

Table of Contents

1. Language Description (C++)	4
2. Selected Functionalities	4
3. Regular Expressions / Rules (Per Functionality)	4
3.1 Lexical Analysis Rules	4
3.2 Keyword Recognition Rule	5
3.3 Semicolon-Free Rule (STMT_END Insertion)	5
3.4 Syntax Grammar Rules	6
3.5 Syntax Error Rules	6
3.6 Semantic Rules	7
4. Source Code (Implementation Overview)	7
Github Link:	7
https://github.com/UnaizaAsif/SafeCPP.git	7
5. Output Description	8
Lexical Phase	8
Syntax Phase	8
Semantic Phase	8
Final Summary	8
6. Limitations	8
7. Assumptions	9
8. Outputs	9
Example Output 1	9
Example Output 2	10
Example Output 3	12

SafeCPP : Static Analyzer

1. Language Description (C++)

C++ is a high-performance, statically typed programming language widely used for system programming, game engines, compilers, and performance-critical applications. It supports procedural, object-oriented, and generic programming paradigms. Due to its low-level memory access capabilities (pointers, manual memory management), C++ is powerful but prone to critical runtime errors such as segmentation faults, memory leaks, and undefined behavior. SafeCPP is designed to detect these issues at compile time using static semantic analysis.

2. Selected Functionalities

SafeCPP implements the following core functionalities:

1. Detection of uninitialized variables (Undefined Behavior)
2. Null pointer dereference detection
3. Memory leak detection
4. Circular include dependency detection
5. Type inference using let
6. Semicolon-optional syntax support
7. Symbol table management
8. Syntax error detection with line/column reporting
9. Lexical tokenization of source code
10. Recognition of keywords, identifiers, literals, operators, and delimiters
11. Comment and whitespace handling

3) Regular Expressions / Rules (Per Functionality)

3.1 Lexical Analysis Rules

- Identifier:

$[A-Za-z][A-Za-z0-9]^*$

- Integer literal:

$[0-9]^+$

- Floating-point literal:

```
[0-9]+\.[0-9]+
```

- String literal:

```
"([^"\\]|\\.)*"
```

- Character:

```
'([^'\\]|\\.).'
```

- Single-line comment:

```
//.*
```

- Multi-line comment:

```
/\*[ \S\S]*?\*/
```

- Operators (multi-character priority):

```
== | != | <= | >= | << | >> | && | || | ->
```

- Operators/delimiters:

```
+ - * / % = ! < > & ( ) { } [ ] ; , . :
```

3.2 Keyword Recognition Rule

If a token matches reserved words such as:

```
int, float, double, char, bool, if, else, while, for, return, let, new,
delete, nullptr, include
```

→ classify as **KEYWORD**, otherwise → **IDENTIFIER**

3.3 Semicolon-Free Rule (STMT_END Insertion)

A virtual statement terminator is inserted when:

- A valid statement-ending token appears (identifier, literal, return, etc.)
- Followed by one or more NEWLINES
- Next meaningful token begins a new statement or block

3.4 Syntax Grammar Rules

- Declaration:

```
<type> <id> [= expression]
```

- Let declaration:

```
• let <id> = expression
```

- Assignment:

```
<id> = expression
```

- If statement:

```
if (expression) statement [else statement]
```

- While loop:

```
while (expression) statement
```

- Function:

```
<type> <id>(params) { body }
```

- Statement termination:

```
; | NEWLINE | STMT_END
```

3.5 Syntax Error Rules

- Missing ;,), {, {, }
- Unexpected tokens in declarations or expressions
- Invalid variable declarations (e.g., int a,,b)
- Empty or incomplete expressions (int a = ;)
- Unexpected operator usage (a == = b)

3.6 Semantic Rules

- **System 1 (Uninitialized usage):**
Detect variables used before assignment
- **System 2 (Null pointer):**
Detect dereference of nullptr or unknown pointer state
- **System 3 (Memory leaks):**
Detect new without delete, including loop-based leaks
- **System 4 (Include analysis):**
Build include graph and detect cycles using DFS
- **System 5 (Let inference):**
 - Integer → int
 - Floating → double
 - String → string
 - Char → char
 - Boolean → bool
 - Else → unknown
- **System 6 (Semicolon-free reporting):**
Reports each virtual statement boundary

4) Source Code (Implementation Overview)

Github Link:

<https://github.com/UnaizaAsif/SafeCPP.git>

- **main.cpp**
Controls execution pipeline (Lexical → Syntax → Semantic)
- **lexer/**
Token generation, classification, and statement termination logic
- **parser/**
Recursive-descent parser with AST construction and error recovery
- **semantic/**
Implements safety systems (1–6), symbol tracking, and type inference

- **utils/**
Symbol table for tracking variable state, initialization, and allocation
- **analyzer/**
Include dependency graph + cycle detection module
- **UI/** (optional)
Express backend + React frontend for visualization

5) Output Description

The compiler produces structured phase-wise output:

Lexical Phase

- Full token stream with type, value, line, column

Syntax Phase

- Syntax errors with precise location
- VALID / INVALID status
- Total error count

Semantic Phase

- System-based warnings:
 - Uninitialized variable usage
 - Null pointer dereference
 - Memory leaks
 - Include cycles
 - Type inference results

Final Summary

- Total safety issues
- Overall program validity status

6) Limitations

- Supports only a limited subset of C++ grammar.
- Uses simplified recursive-descent parsing.
- No complete AST-based semantic analysis.
- Complex expressions are not fully handled.
- Operator precedence handling is limited.

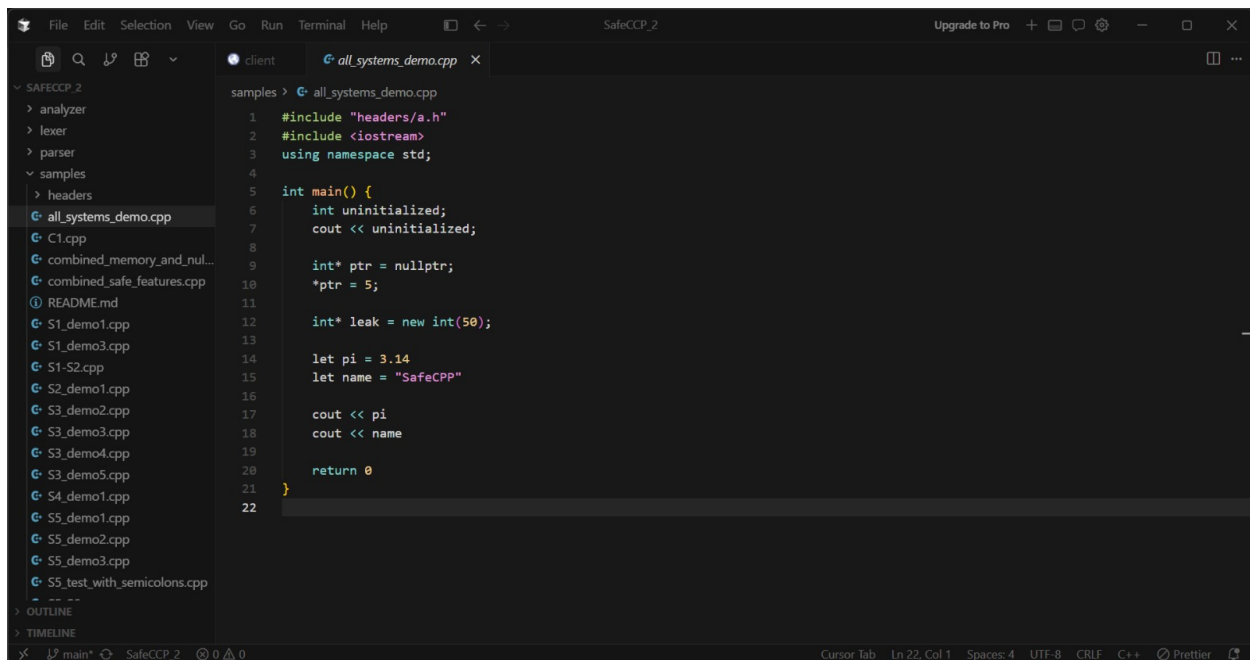
- Loop and control-flow analysis is partial.
- Semantic analysis is mostly token-based.
- Include analysis supports only "file.h" style includes.
- Include checking does not fully verify file existence.
- Syntax recovery may skip deeper errors.
- No runtime checking or code execution is performed.
- No compiler backend or code generation is included.

7) Assumptions

- Input code is written in a simplified C++-like syntax supported by SafeCPP.
- Code is processed in a single pass using tokens (no full execution).
- Variables follow block scope using { } braces.
- Semicolon is optional (newline or STMT_END can end a statement).
- Only new and delete are tracked for memory management.
- Only #include "file.h" style includes are considered for dependency analysis.
- Type inference using let assumes RHS expression is enough to determine type.
- The system performs static analysis only (no runtime execution)

8) Outputs

Example Output 1



The screenshot shows the SafeCPP 2 IDE interface. On the left is a file explorer with a tree view containing folders like 'analyzer', 'lexer', 'parser', and 'samples', along with various demo files. The main editor window displays the file 'all_systems_demo.cpp' with the following code:

```

1  #include "headers/a.h"
2  #include <iostream>
3  using namespace std;
4
5  int main() {
6      int uninitialized;
7      cout << uninitialized;
8
9      int* ptr = nullptr;
10     *ptr = 5;
11
12     int* leak = new int(50);
13
14     let pi = 3.14
15     let name = "SafeCPP"
16
17     cout << pi
18     cout << name
19
20     return 0
21 }
22

```

The status bar at the bottom indicates 'Cursor Tab', 'Ln 22, Col 1', 'Spaces: 4', 'UTF-8', 'CRLF', 'C++', and 'Prettier'.

The screenshot shows the 'SafeCPP Visualizer' application in a web browser. The file 'all_systems_demo.cpp' is loaded. The 'Run Analysis' button is visible. The interface is split into two main panels: 'LEXICAL TOKENS' on the left and 'SYNTAX ANALYSIS' on the right.

LEXICAL TOKENS

Token stream:

Token-1 - '#' at Line 1, Column 1
 <
 Token: "UNKNOWN"
 Lexeme: "#"
 Location: Line 1, Column 1
 >

Token-2 - "include" at Line 1, Column 2
 <
 Token: "INCLUDE"
 Lexeme: "include"
 Location: Line 1, Column 2
 >

Token-3 - "headers/a.h" at Line 1, Column 10
 <
 Token: "STRING"
 Lexeme: "headers/a.h"
 Location: Line 1, Column 10
 >

Token-4 - "\n" at Line 1, Column 23
 <
 Token: "NEWLINE"
 Lexeme: "\n"
 Location: Line 1, Column 23
 >

SYNTAX ANALYSIS

Syntax issues found:
 Parse error at line 12, column 17: Syntax Error: Unexpected token 'new'. Expected: identifier or number

[SYNTAX SUMMARY]
 SYNTAX STATUS: INVALID
 TOTAL SYNTAX ISSUES: 1

The screenshot shows the 'SafeCPP Visualizer' application with the 'Run Analysis' button. The interface is split into two main panels: 'SEMANTIC ANALYSIS' on the left and 'SUMMARY' on the right.

SEMANTIC ANALYSIS

[SEMANTIC ERROR]
 SYSTEM 4: INCLUDE DEPENDENCY ANALYSIS
 Circular include dependency found.
 Risk: Headers including each other create circular dependencies.
 Impact: Compilation may fail or create infinite include chains.
 Cycle Path: c.h → a.h → b.h → c.h
 Location: Line 1, Column 0

[SEMANTIC ERROR]
 SYSTEM 1: UNDEFINED BEHAVIOR DETECTION
 Variable 'uninitialized' was never initialized.
 Risk: Reading garbage memory leads to unpredictable behavior.
 Location: Line 7, Column 13

[SEMANTIC ERROR]
 SYSTEM 2: NULL POINTER SAFETY
 Pointer 'ptr' dereferenced without null check..
 Risk: System Crash / Segmentation fault / Memory access violation.
 Location: Line 10, Column 6

[SEMANTIC INFO]
 TYPE_INFERRED_DOUBLE:
 Variable 'pi' inferred as type 'double'

[SEMANTIC INFO]
 TYPE_INFERRED_STRING:
 Variable 'name' inferred as type 'string'

[SYNTAX INFO]
 ...

SUMMARY

Total safety issues: 4

Example Output 2

The screenshot shows a code editor with a file named `sy1.cpp` open. The code contains several syntax errors:

```

1  #include <iostream
2  using namespace std
3
4  int main( {
5
6      int x
7      int b,10;
8
9      if x > 5 {
10         cout << x
11     }
12
13     return 0
14

```

The errors are highlighted in the editor's status bar at the bottom: "syntax invalid plus after eq..." and "OUTLINE".

The screenshot shows the **SafeCPP Visualizer** web application. The file `sy1.cpp` is loaded, and the **Run Analysis** button has been clicked. The application displays the following analysis results:

LEXICAL TOKENS

Token stream:

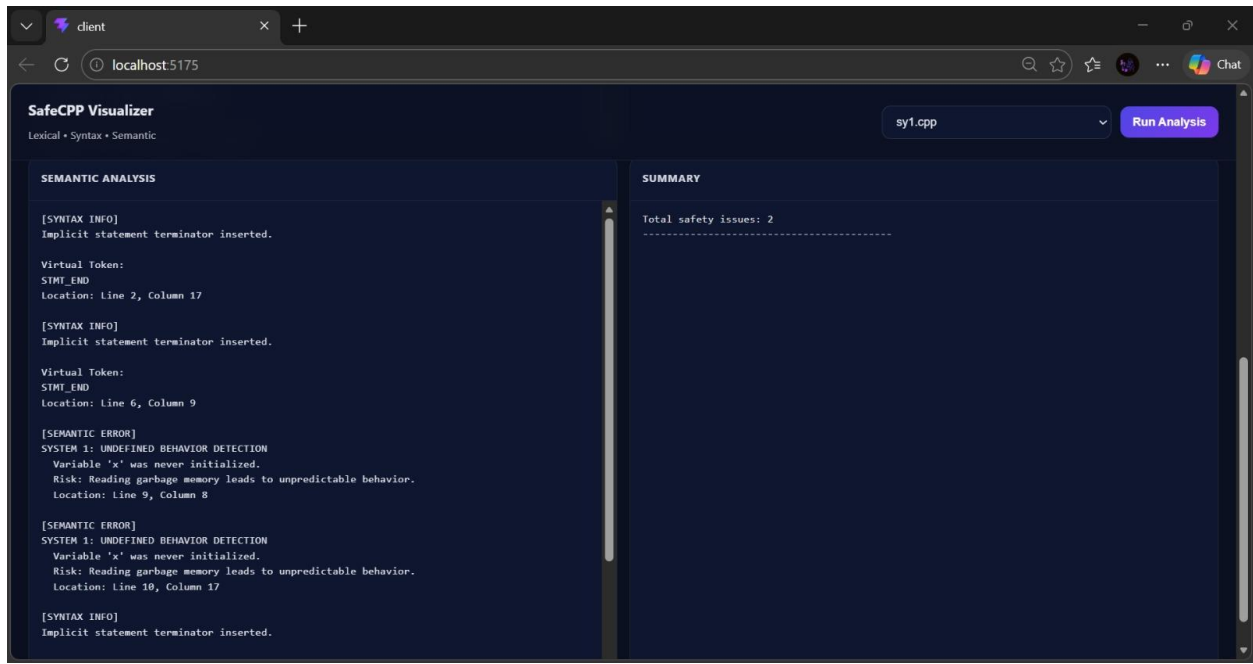
- Token-1 - "#" at Line 1, Column 1
Token: "UNKNOWN"
Lexeme: "#"
Location: Line 1, Column 1
- Token-2 - "include" at Line 1, Column 2
Token: "INCLUDE"
Lexeme: "include"
Location: Line 1, Column 2
- Token-3 - "<" at Line 1, Column 10
Token: "LESS_THAN"
Lexeme: "<"
Location: Line 1, Column 10
- Token-4 - "iostream" at Line 1, Column 11
Token: "IDENTIFIER"
Lexeme: "iostream"
Location: Line 1, Column 11

SYNTAX ANALYSIS

Syntax issues found:

- Parse error at line 4, column 12: Expected parameter name
- Parse error at line 7, column 10: Invalid declarator after ',' in variable declaration
- Parse error at line 9, column 8: Expected LEFT_PAREN but got IDENTIFIER
- Parse recovery warning at line 11, column 5: forced token advance to avoid parser stall.
- Parse recovery warning at line 11, column 6: forced token advance to avoid parser stall.

[SYNTAX SUMMARY]
SYNTAX STATUS: INVALID
TOTAL SYNTAX ISSUES: 5



Example Output 3

